UNIVERSIDAD DE LAS FUERZAS ARMADAS

ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INVESTIGACIÓN EN LA

INGENIERÍA DE SOFTWARE

INGENIERÍA DE SOFTWARE

Integrantes: Yostin Sisalema

Josué Chango

Docente: Rea Guaman Angel Marcelo

NRC: 30821

Fecha: 16 de abril del 2026

Vulnerabilidades introducidas por código generado con IA: ¿los desarrolladores las detectan?

1. Definición del tema

El tema de investigación se centra en las vulnerabilidades de seguridad que los modelos de lenguaje grande (LLM) introducen durante la generación automática de código fuente, y en la capacidad real que tienen los desarrolladores para identificarlas antes de que ese código llegue a producción.

Herramientas como GitHub Copilot, Amazon CodeWhisperer, Tabnine o Cursor se han convertido en asistentes cotidianos para muchos equipos de desarrollo. Generan fragmentos de código a partir de instrucciones en lenguaje natural y del contexto del proyecto, lo que acelera el trabajo. Sin embargo, ese código no siempre cumple estándares mínimos de seguridad: puede contener patrones inseguros heredados del entrenamiento, ausencia de validación de entradas, manejo incorrecto de valores aleatorios criptográficos o exposición de información sensible.

Lo que hace relevante el problema no es solo la existencia de vulnerabilidades, sino la cadena humana que las recibe: los desarrolladores que aceptan sugerencias de IA sin revisarlas a fondo, los equipos sin herramientas de análisis estático en su flujo de trabajo, y las organizaciones que desconocen que parte de su código proviene de modelos entrenados sobre repositorios que también contenían código defectuoso. La pregunta de investigación no es si existe el problema, sino si quienes trabajan con estas herramientas lo perciben y qué hacen al respecto.

2. Justificación del tema

La adopción de asistentes de código basados en IA ha crecido de forma sostenida desde la aparición de herramientas accesibles al público general. Este crecimiento no ha venido acompañado de una madurez equivalente en las prácticas de revisión de seguridad. La brecha entre la velocidad con que se incorpora código generado y la velocidad con que se audita representa un riesgo concreto para la integridad de los sistemas.

Investigaciones recientes han encontrado que una fracción significativa de los fragmentos de código producidos por estos modelos en proyectos reales presenta debilidades clasificadas dentro del sistema CWE (Common Weakness Enumeration), incluyendo categorías reconocidas como críticas por MITRE en su lista anual Top-25. Entre las más frecuentes aparecen el uso de valores aleatorios insuficientemente seguros, la inyección de código y el Cross-site Scripting —no son problemas teóricos, sino patrones que pueden comprometer autenticación, sesiones y validación de entradas en aplicaciones reales.

Estudiar si los desarrolladores detectan estas vulnerabilidades aporta valor en dos direcciones. Por un lado, permite entender si el problema es de conocimiento —el desarrollador no sabe identificar ese tipo de fallo— o de proceso —sí lo conoce, pero la confianza en la herramienta reduce la atención que le dedica. Por otro lado, los hallazgos

pueden orientar el diseño de flujos de revisión más seguros y la formación de competencias en programadores que cada vez dependen más de estos sistemas.

El tema es pertinente para la Ingeniería de Software porque toca directamente la calidad del producto, la fiabilidad de los procesos de desarrollo y la responsabilidad del ingeniero frente al código que entrega, independientemente de quién —o qué— lo escribió primero.

3. Qué se desea conseguir

Objetivo general

Determinar si los desarrolladores de software identifican las vulnerabilidades de seguridad introducidas por modelos de lenguaje grande durante la generación de código, analizando tanto la frecuencia y categoría de las debilidades producidas como los factores del proceso de desarrollo que facilitan o dificultan su detección.

Objetivos específicos

- Identificar y clasificar, usando el sistema CWE, las categorías de vulnerabilidades más frecuentes en código generado por diferentes modelos LLM (Copilot, ChatGPT, CodeWhisperer) a partir de fragmentos producidos ante prompts equivalentes, distinguiendo resultados por lenguaje de programación y tipo de tarea.
- Evaluar cómo la inclusión de requisitos de seguridad explícitos en el prompt de entrada modifica la frecuencia y gravedad de las vulnerabilidades generadas, comparando código producido con instrucciones neutras frente a código producido con instrucciones que especifican restricciones de seguridad.
- Establecer qué herramientas de análisis estático y qué prácticas de revisión de código —revisión manual, pruebas automatizadas, revisión en par— permiten detectar con mayor eficacia las vulnerabilidades introducidas por código generado con IA en un contexto de desarrollo real o simulado.